

Лабораторная работа №5

Проектирование архитектуры ПО с использованием нотации C4 и PlantUML

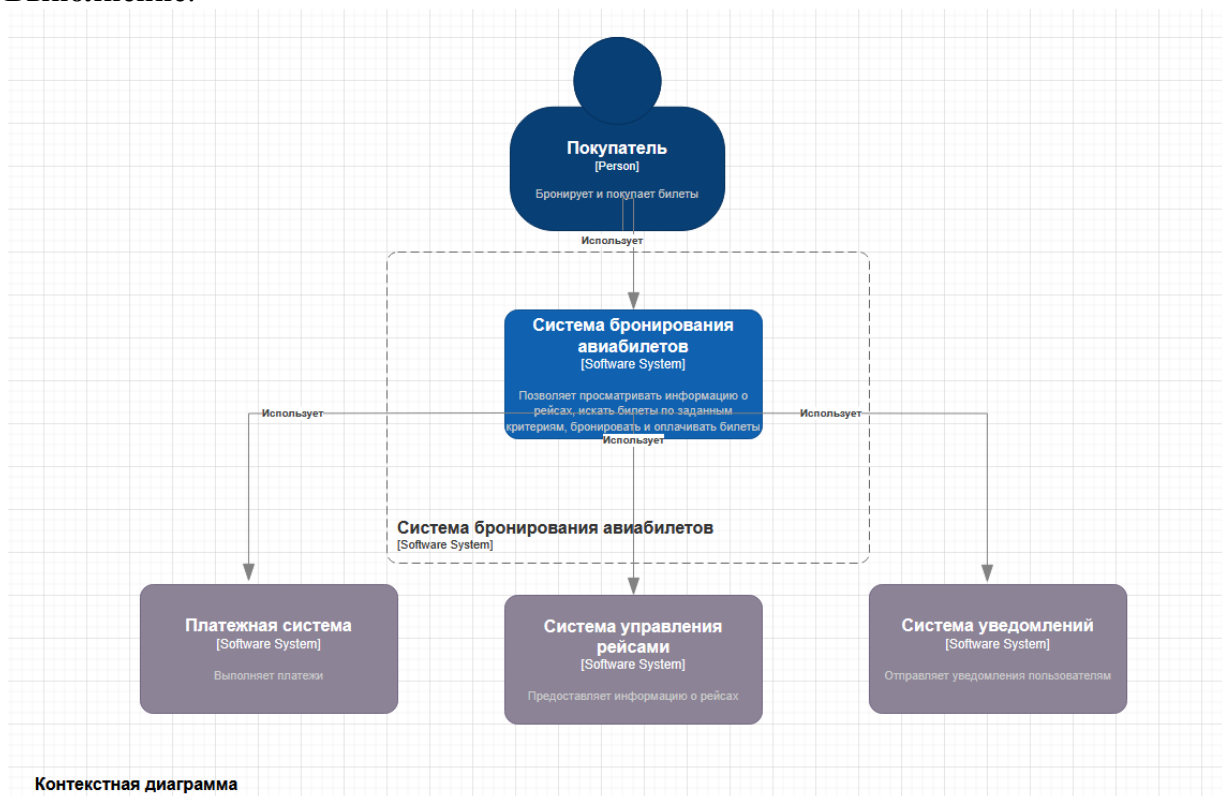
Цель работы: Освоить применение нотации C4 для визуализации архитектуры программного обеспечения.

Задание 1. Необходимо разработать систему бронирования авиабилетов для небольшой авиакомпании. Система должна позволять пользователям:

- Поиск авиабилетов по заданным критериям (город отправления, город прибытия, дата, количество пассажиров).
- Просмотр информации о рейсах (время отправления, время прибытия, стоимость, количество свободных мест).
- Бронирование авиабилетов.
- Оплату авиабилетов.
- Просмотр информации о забронированных билетах.
- Отмену бронирования.

Задание 1.1. Используя онлайн редактор draw.io (<https://app.diagrams.net/>) построить контекстную диаграмму для Системы бронирования авиабилетов.

Выполнение:



Задание 1.2. Используя онлайн редактор PlantText (<https://www.planttext.com/>) спроектировать архитектуру приложения, используя нотацию C4, т.е. построить

1. контекстную диаграмму,
2. диаграмму контейнеров,
3. диаграмму компонентов
4. диаграмму классов
5. диаграмму последовательностей

Выполнение

Этап 1: Контекстная диаграмма

1. Определим систему, которую будем проектировать: Система бронирования авиабилетов.
2. Определить пользователей системы (актеров): Покупатель.

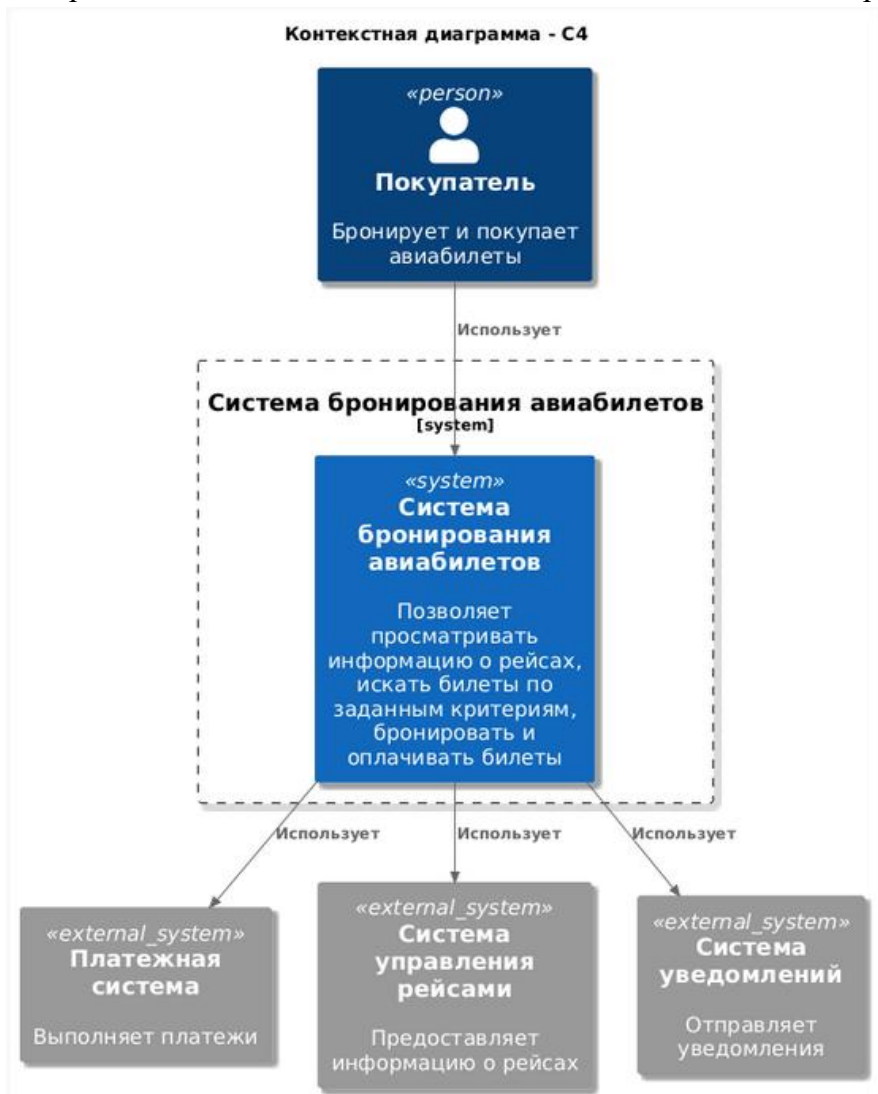
3. Определим внешние системы, с которыми будет взаимодействовать проектируемая система :платежная система, система управления рейсами авиакомпании, система уведомлений.
4. Создадим контекстную диаграмму, используя нотацию C4 и PlantUML.

Пример кода PlantUML для контекстной диаграммы:

```
@startuml
title Контекстная диаграмма - C4
!include <c4/C4_Context>
Person(customer, "Покупатель", "Бронирует и покупает авиабилеты")
System_Boundary(c1, "Система бронирования авиабилетов") {
    System(booking_system, "Система бронирования авиабилетов", "Позволяет просматривать информацию о рейсах, искать билеты по заданным критериям, бронировать и оплачивать билеты")
}
System_Ext(Payment_Gateway, "Платежная система", "Выполняет платежи")
System_Ext(Flight_Management_System, "Система управления рейсами", "Предоставляет информацию о рейсах")
System_Ext(Notification_Service, "Система уведомлений", "Отправляет уведомления")
Rel(customer, booking_system, "Использует")
Rel(booking_system, Payment_Gateway, "Использует")
Rel(booking_system, Flight_Management_System, "Использует" )
Rel(booking_system, Notification_Service, "Использует")

skinparam {
    DefaultFontSize 16
    shadowing true
}
@enduml
```

Построенная в соответствии с данным кодом контекстная диаграмма будет выглядеть так:



Этап 2: Диаграмма контейнеров

1. Определим контейнеры, составляющие систему бронирования авиабилетов. Контейнеры – это независимо развертываемые единицы, такие как веб-приложения, мобильные приложения, базы данных, API. Пусть в нашем случае будут определены следующие контейнеры: веб-приложение, API, база данных, очередь сообщений).
2. Для каждого контейнера укажем его тип, технологию и назначение.
3. Создадим диаграмму контейнеров, используя нотацию C4 и PlantUML .

Пример кода PlantUML для диаграммы контейнеров:

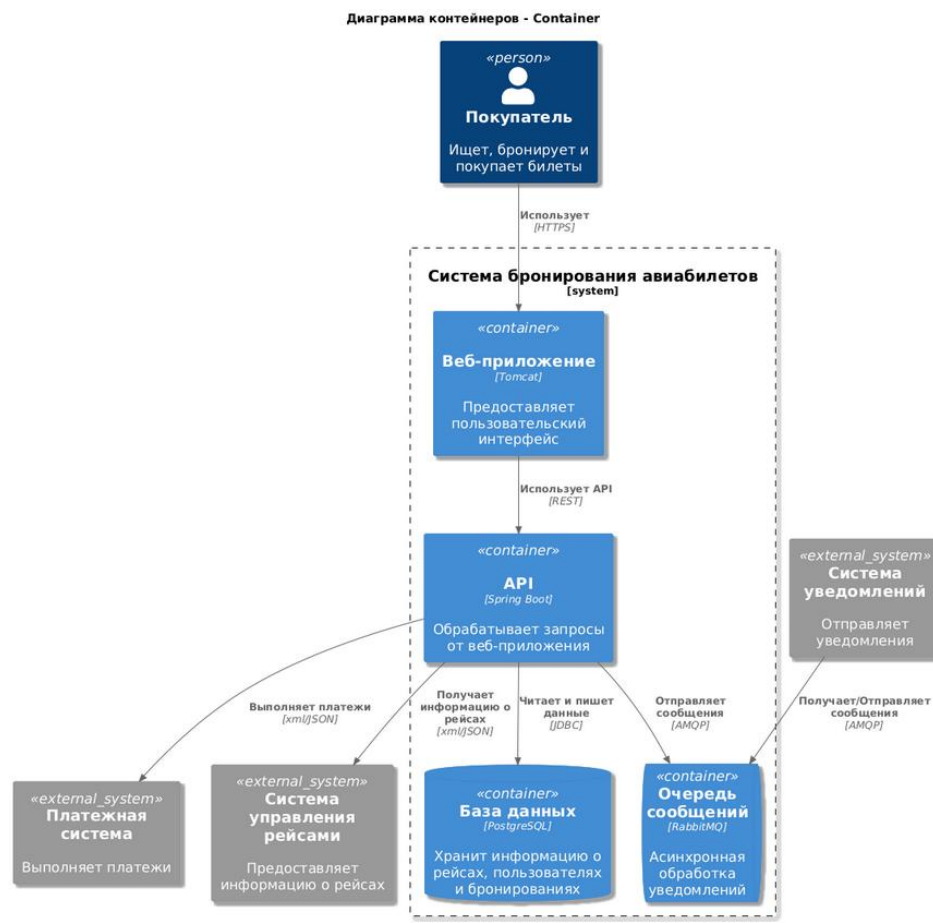
```
@startuml
title Диаграмма контейнеров - Container
!include <c4/C4_Container>
Person(customer, "Покупатель", "Ищет, бронирует и покупает билеты")

System_Boundary(c1, "Система бронирования авиабилетов") {
    Container(web_app, "Веб-приложение", "Tomcat", "Предоставляет пользовательский интерфейс", "HTTPS")
    Container(api, "API", "Spring Boot", "Обработывает запросы от веб-приложения", "REST")
    ContainerDb(database, "База данных", "PostgreSQL", "Хранит информацию о рейсах, пользователях и бронированиях")
    ContainerQueue(message_queue, "Очередь сообщений", "RabbitMQ", "Асинхронная обработка уведомлений")
}

System_Ext(Notification_Service, "Система уведомлений", "Отправляет уведомления")
System_Ext(Payment_Gateway, "Платежная система", "Выполняет платежи")
System_Ext(Flight_Managment_System, "Система управления рейсами", "Предоставляет информацию о рейсах")

Rel(web_app, api, "Использует API", "REST")
Rel(api, database, "Читает и пишет данные", "JDBC")
Rel(api, message_queue, "Отправляет сообщения", "AMQP")
Rel(Notification_Service, message_queue, "Получает/Отправляет сообщения", "AMQP")
Rel(customer, web_app, "Использует", "HTTPS")
Rel(api, Flight_Managment_System, "Получает информацию о рейсах", "xml/JSON")
Rel(api, Payment_Gateway, "Выполняет платежи", "xml/JSON")
skinparam {
    DefaultFontSize 16
    shadowing true
}
@enduml
```

Построенная в соответствии с данным кодом диаграмма контейнеров будет выглядеть так:



Этап 3: Диаграмма компонентов

1. Построим диаграмму компонентов для контейнера Веб-приложение.
2. Определим компоненты, составляющие контейнер : модуль поиска рейсов, модуль бронирования, модуль оплаты, модуль управления пользователями.
3. Создадим диаграмму компонентов, используя нотацию C4 и

Пример кода PlantUML для диаграммы компонентов (для веб-приложения):

```
@startuml
title Диаграмма компонентов
!include <c4/C4_Component>

Container(web_app, "Веб-приложение", "Tomcat", "Предоставляет пользовательский интерфейс",
"HTTPS") {
    Component(search_module, "Модуль поиска рейсов", "Позволяет пользователям искать рейсы
по заданным критериям")
    Component(booking_module, "Модуль бронирования", "Позволяет пользователям бронировать
авиабилеты")
    Component(payment_module, "Модуль оплаты", "Обрабатывает платежи пользователей")
    Component(user_management_module, "Модуль управления пользователями", "Позволяет
пользователям регистрироваться, входить в систему и управлять своими данными")
}
Rel(web_app, search_module, "Использует")
Rel(web_app, booking_module, "Использует")
Rel(web_app, payment_module, "Использует")
Rel(web_app, user_management_module, "Использует")

skinparam {
    DefaultFontSize 16
    shadowing true
}

@enduml
```

Построенная в соответствии с данным кодом диаграмма компонентов будет выглядеть так:



Этап 4. Диаграмма классов для Модуля поиска билетов

Пример кода PlantUML для диаграммы классов:

```
@startuml

title Диаграмма классов (Модуль поиска билетов)- Class

skinparam {
  DefaultFontSize 15
}

class SearchModule {
  - flightRepository: FlightRepository
  - airportRepository: AirportRepository
  + searchFlights(searchCriteria: SearchCriteria): List<Flight>
}

class SearchCriteria {
  - originAirportCode: String
  - destinationAirportCode: String
  - departureDate: LocalDate
  - returnDate: LocalDate
  - numberOfPassengers: int
  + isValid(): boolean
}

class Flight {
  - flightNumber: String
  - originAirport: Airport
  - destinationAirport: Airport
  - departureDateTime: LocalDateTime
  - arrivalDateTime: LocalDateTime
  - price: BigDecimal
  - availableSeats: int
  + isAvailable(): boolean
}

class Airport {
  - airportCode: String
  - airportName: String
  - city: String
  - country: String
}

interface FlightRepository {
  + findFlights(searchCriteria: SearchCriteria): List<Flight>
}

interface AirportRepository {
  + getAirportByCode(airportCode: String): Airport
}

SearchModule -- FlightRepository : uses
SearchModule -- AirportRepository : uses
SearchModule -- SearchCriteria : uses
SearchModule -- Flight : returns
Flight -- Airport : originAirport
Flight -- Airport : destinationAirport

note top of SearchModule : Модуль поиска авиабилетов по заданным критериям

note top of SearchCriteria : Содержит критерии поиска (город отправления, \nгород прибытия
, дата, количество пассажиров)

note top of Flight : Представляет информацию о рейсе

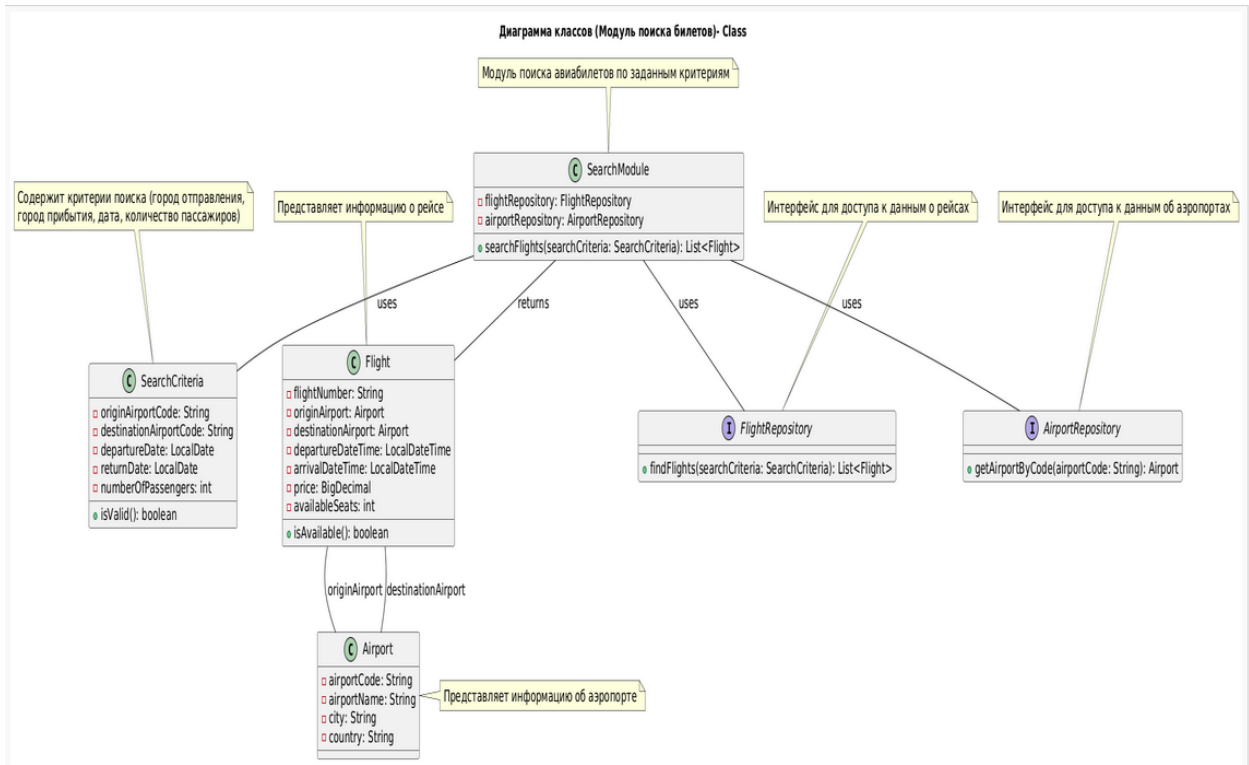
note right of Airport : Представляет информацию об аэропорте

note top of FlightRepository : Интерфейс для доступа к данным о рейсах

note top of AirportRepository : Интерфейс для доступа к данным об аэропортах

@enduml
```

Построенная в соответствии с данным кодом диаграмма классов будет выглядеть так:



Описание классов

Имя класса	Атрибуты	Операции
SearchModule (основной класс модуля поиска)	flightRepository airportRepository	searchFlights(searchCriteria: SearchCriteria): List<Flight> (Метод для выполнения поиска рейсов на основе заданных критериев)
SearchCriteria (Класс, представляющий критерии поиска)	originAirportCode (Код аэропорта отправления) destinationAirportCode (Код аэропорта прибытия) departureDate (дата отправления) returnDate (дата возвращения) numberOfPassengers (количество пассажиров)	isValid(): boolean (Метод для проверки валидности критериев поиска)
Flight (Класс, представляющий информацию о рейсе)	originAirport (Аэропорт отправления) destinationAirport (Аэропорт прибытия) departureDateTime (Дата и время отправления) arrivalDateTime (Дата и время прибытия) Price (Цена билета) availableSeats (Количество доступных мест)	isAvailable(): boolean (Метод для проверки доступности мест на рейсе)
Airport (Класс, представляющий информацию об аэропорте)	airportCode (Код аэропорта) airportName (Наименование аэропорта) City (Город) Country (Страна)	

FlightRepository (Интерфейс для доступа к данным о рейсах)		findFlights(searchCriteria: SearchCriteria): List<Flight> (Метод для поиска рейсов на основе заданных критериев)
AirportRepository (Интерфейс для доступа к данным об аэропортах)		getAirportByCode(airportCode: String): Airport (Метод для получения информации об аэропорте по его коду)

Этап 5. Диаграмма последовательностей

Пример кода PlantUML для диаграммы последовательностей

```
@startuml
skinparam {
defaultfontsize 16
}
title Диаграмма последовательности
actor User
participant SearchModule
participant FlightRepository
participant AirportRepository
participant Database

User -> SearchModule: Запрос на поиск авиабилетов (originAirportCode,
destinationAirportCode, departureDate, returnDate, numberOfPassengers)

activate SearchModule
SearchModule -> SearchModule: Создание объекта SearchCriteria

alt Валидные критерии поиска
SearchModule -> AirportRepository: getAirportByCode(originAirportCode)
activate AirportRepository
AirportRepository -> Database: SELECT * FROM Airports WHERE airportCode =
originAirportCode
activate Database
Database --> AirportRepository: Возвращает Airport (originAirport) или null
deactivate Database
AirportRepository --> SearchModule: Возвращает Airport (originAirport) или null
deactivate AirportRepository

SearchModule -> AirportRepository: getAirportByCode(destinationAirportCode)
activate AirportRepository
AirportRepository -> Database: SELECT * FROM Airports WHERE airportCode =
destinationAirportCode
activate Database
Database --> AirportRepository: Возвращает Airport (destinationAirport) или null
deactivate Database
AirportRepository --> SearchModule: Возвращает Airport (destinationAirport) или null
deactivate AirportRepository

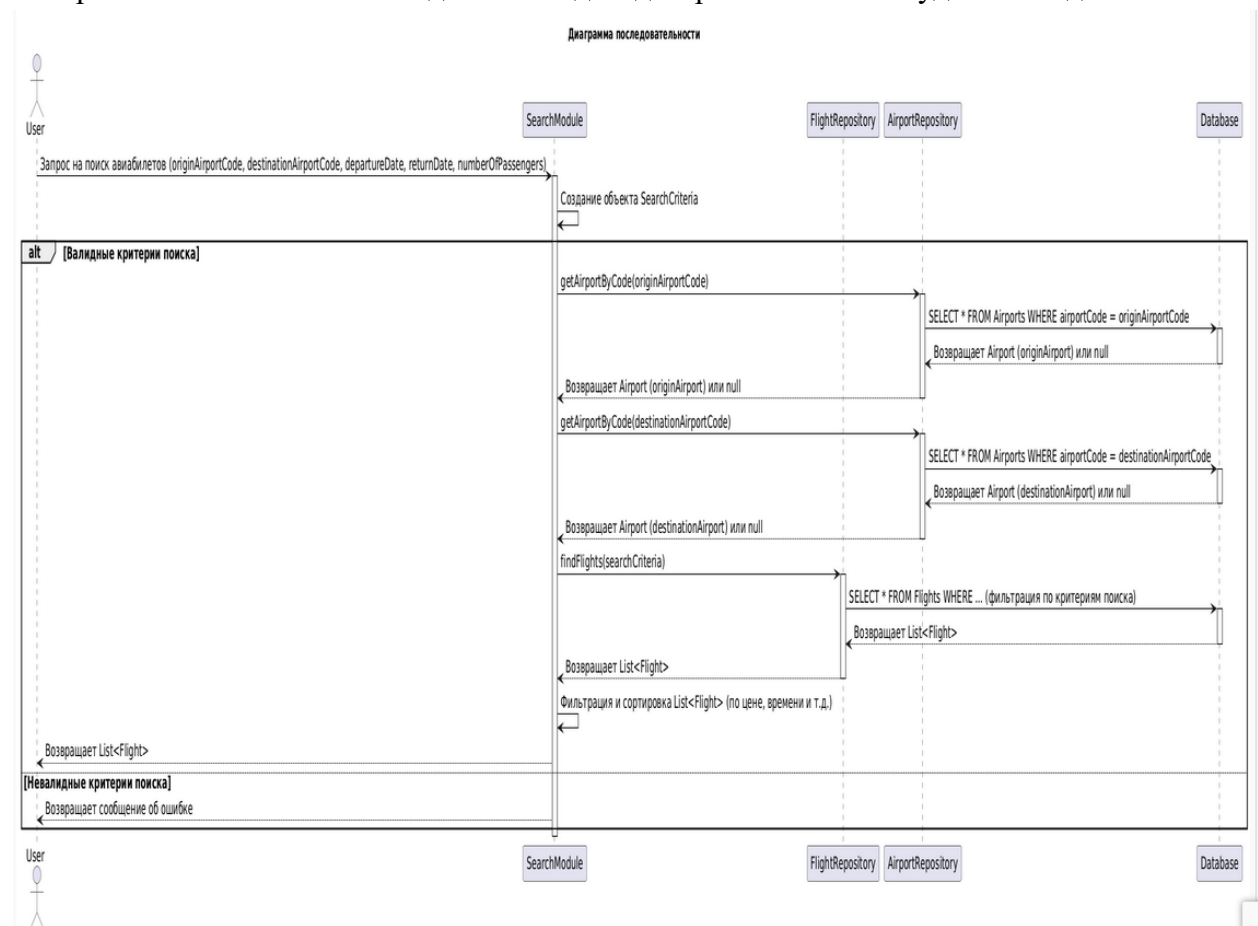
SearchModule -> FlightRepository: findFlights(searchCriteria)
activate FlightRepository
FlightRepository -> Database: SELECT * FROM Flights WHERE ... (фильтрация по критериям
поиска)
activate Database
Database --> FlightRepository: Возвращает List<Flight>
deactivate Database
FlightRepository --> SearchModule: Возвращает List<Flight>
deactivate FlightRepository

SearchModule -> SearchModule: Фильтрация и сортировка List<Flight> (по цене, времени и т
.д.)

SearchModule --> User: Возвращает List<Flight>

else Невалидные критерии поиска
SearchModule --> User: Возвращает сообщение об ошибке
end
deactivate SearchModule
@enduml
```

Построенная в соответствии с данным кодом диаграмма классов будет выглядеть так:



Описание диаграммы:

1. **User:** Актер, представляющий пользователя, который инициирует поиск авиабилетов.
2. **Объекты:**
 - **SearchModule:** Объект, отвечающий за обработку запроса на поиск авиабилетов.
 - **FlightRepository:** Объект, отвечающий за доступ к данным о рейсах (например, через базу данных).
 - **AirportRepository:** Объект, отвечающий за доступ к данным об аэропортах.
 - **Database:** Представляет базу данных, содержащую информацию о рейсах и аэропортах.

Последовательность действий:

1. **Запрос на поиск:** Пользователь отправляет запрос на поиск авиабилетов в SearchModule с указанием критериев поиска (город отправления, город прибытия, дата отправления, дата возвращения, количество пассажиров).
2. **Создание объекта SearchCriteria:** SearchModule создает объект SearchCriteria, содержащий критерии поиска.
3. **Проверка критериев:**

Альтернативный блок: Если критерии поиска валидны, выполняется следующий шаг, иначе, SearchModule возвращает сообщение об ошибке пользователю.
5. **Получение информации об аэропортах:** SearchModule использует AirportRepository, чтобы получить информацию об аэропортах отправления и прибытия из базы данных.
 - a. AirportRepository отправляет запросы в Database для получения информации об аэропортах по коду.
 - b. Database возвращает информацию об аэропортах или null, если аэропорт не найден.
 - c. AirportRepository возвращает информацию об аэропорте или null в SearchModule.
6. **Поиск рейсов:** SearchModule использует FlightRepository, чтобы найти рейсы, соответствующие критериям поиска.

- a. FlightRepository отправляет запрос в Database с критериями поиска (фильтрация по городу отправления, городу прибытия, дате и т.д.).
 - b. Database возвращает список рейсов, соответствующих критериям.
 - c. FlightRepository возвращает список рейсов в SearchModule.
7. **Фильтрация и сортировка:** SearchModule фильтрует и сортирует список рейсов (например, по цене, времени и т.д.).
8. **Возврат результатов:** SearchModule возвращает список найденных рейсов пользователю.

Справка

Основные элементы С4 и их синтаксис

Элемент	Синтаксис	Параметры
Person (Пользователь)	Person(Alias, Name, Description) Person_Ext(Alias, Name, Description) ' Внешний пользователь	Alias: Уникальный идентификатор элемента. Name: Отображаемое имя элемента.
Software System (Программная система)	System(Alias, Name, Description) System_Ext(Alias, Name, Description) ' Внешняя система	Description: Описание элемента.
Container (Контейнер)	Container(Alias, Name, Technology, Description) ContainerDb(Alias, Name, Technology, Description) ' Контейнер-база данных ContainerQueue(Alias, Name, Technology, Description) ' Контейнер-очередь сообщений	Alias: Уникальный идентификатор элемента. Name: Отображаемое имя элемента. Technology: Используемая технология
Component (Компонент)	Component(Alias, Name, Technology, Description)	Description: Описание элемента
Relationship (Отношение):	Rel(SourceAlias, DestinationAlias, Description, Technology) Rel_D(SourceAlias, DestinationAlias, Description, Technology) ' Пунктирная линия	SourceAlias: Идентификатор элемента-источника. DestinationAlias: Идентификатор элемента-назначения. Description: Описание отношения. Technology: Используемая технология/протокол